

CASE STUDY SCENARIO

A multi-specialty hospital is planning to modernize its Hospital Management System (HMS) to efficiently manage different types of patients such as Inpatients, Outpatients, and Emergency Patients. To improve system performance and scalability, the hospital wants to redesign the system using advanced Object-Oriented Programming concepts in C++.

Case Study Task

You are required to analyze, design, implement, and evaluate a C++ application using:

- Inheritance (types of inheritance)
- Virtual base classes
- Abstract classes
- Constructors in derived classes
- Member classes (composition)
- Pointers and pointers to objects
- this pointer
- Pointer to derived classes
- Runtime polymorphism

ANSWERS:

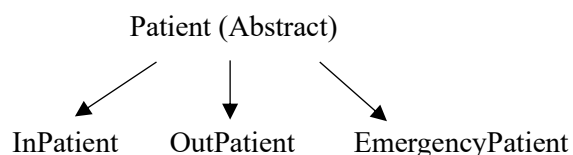
Part A:

1. Identify the limitations of the existing system.
 - Data duplication for patients
 - Rigid billing system
 - Hard to add new patient types
 - Poor memory management
2. Explain how OOP principles can solve these issues.
 - Encapsulation → organizes patient data, reduces redundancy
 - Inheritance → avoids code duplication, improves reusability
 - Polymorphism → allows flexible billing for different patient types
 - Abstraction → hides complexity, shows only necessary details

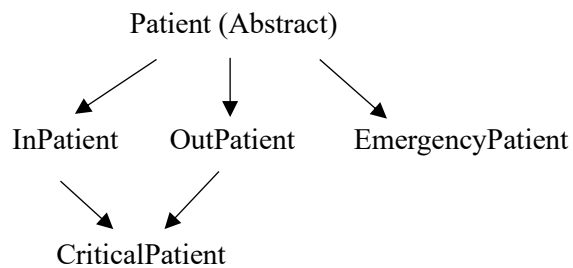
3. List the functional requirements of the improved system.

Part B:

1. Design a class hierarchy:



2. Extend using multiple inheritance:



3. Use:

- o Virtual base classes to resolve ambiguity
- o A member class (Doctor) inside Patient

Part C:

1. Compare:

- Inheritance vs Composition in patient management

Inheritance	Composition
Represents IS-A relationship	Represents HAS-A relationship
Example: InPatient is a Patient	Example: Patient has a Doctor
Promotes code reuse	Improves modularity and flexibility
Easier to extend system with new patient types	Changes in one class do not affect others

- Static vs Dynamic binding

Static	Dynamic
Happens at compile-time	Happens at run-time
Faster execution	Uses virtual functions
Used in normal functions	More flexible and adaptable
Less flexible	Allows correct function call based on object type

2. Analyze:

- o The role of virtual base classes in diamond problem
 - In multiple inheritance, the same base class can be inherited multiple times
 - This creates duplicate copies of base class data
 - Leads to ambiguity and memory waste

o Benefits of pointers in dynamic patient handling

- Allows dynamic memory allocation
- Objects created only when needed
- Supports runtime polymorphism
- Helps manage multiple patients efficiently
- Reduces memory wastage

3. Justify:

o Your design approach

- Uses OOP principles for better structure
- Reduces redundancy using inheritance
- Improves flexibility using polymorphism
- Ensures scalability for future patient types
- Simplifies maintenance and updates

o How it improves scalability and maintainability

Scalability

- New patient types can be added easily
- No need to modify existing code

Maintainability

- Code is modular and well-organized
- Easy to debug and update
- Changes in one class do not affect entire system

Part D:

Develop a C++ program that:

```
#include <iostream>
```

```
using namespace std;
```

```
// Member Class
```

```
class Doctor {
```

```
public:
```

```
    string docName;
```

```
    void setDoctor() {
```

```
        cout << "Enter Doctor Name: ";
```

```
        cin >> docName;
```

```

    }
};

// Abstract Class
class Patient {
protected:
    string name;
    Doctor d;

public:
    virtual void getData() {
        cout << "Enter Patient Name: ";
        cin >> name;
        d.setDoctor();
    }

    virtual void calculateBill() = 0; // pure virtual
    virtual ~Patient() {}
};

// InPatient
class InPatient : virtual public Patient {
protected:
    int room, treatment;

public:
    void getData() {
        Patient::getData();
        cout << "Room Charge: ";
        cin >> room;
        cout << "Treatment Cost: ";
        cin >> treatment;
    }
};

```

```

    }

    int getBill() {
        return this->room + this->treatment; // using this pointer
    }

    void calculateBill() {
        cout << "Bill = " << getBill() << endl;
    }
};

```

// OutPatient

```
class OutPatient : virtual public Patient {
```

```
protected:
```

```
    int consult;
```

```
public:
```

```
    void getData() {
```

```
        Patient::getData();
```

```
        cout << "Consultation Fee: ";
```

```
        cin >> consult;
```

```
    }
```

```
    void calculateBill() {
```

```
        cout << "Bill = " << this->consult << endl;
```

```
    }
```

```
};
```

// EmergencyPatient

```
class EmergencyPatient : virtual public Patient {
```

```
protected:
```

```
    int emergency;
```

public:

```
void getData() {  
    Patient::getData();  
    cout << "Emergency Charge: ";  
    cin >> emergency;  
}
```

```
int getEmergency() {  
    return emergency;  
}
```

```
void calculateBill() {  
    cout << "Bill = " << emergency << endl;  
}
```

};

// Multiple Inheritance

```
class CriticalPatient : public InPatient, public EmergencyPatient {
```

public:

```
void getData() {  
    Patient::getData();  
  
    cout << "Room Charge: ";  
    cin >> room;  
    cout << "Treatment Cost: ";  
    cin >> treatment;  
    cout << "Emergency Charge: ";  
    cin >> emergency;  
}
```

```
void calculateBill() {
```

```

        int total = this->room + this->treatment + this->emergency;

        cout << "Total Bill = ₹" << total << endl;
    }
};

// MAIN

int main() {
    Patient *p;
    int choice;

    cout << "\n1.InPatient\n2.OutPatient\n3.Emergency\n4.Critical\n";
    cout << "Enter choice: ";
    cin >> choice;

    switch(choice) {
        case 1: p = new InPatient(); break;
        case 2: p = new OutPatient(); break;
        case 3: p = new EmergencyPatient(); break;
        case 4: p = new CriticalPatient(); break;
        default: cout << "Invalid"; return 0;
    }

    p->getData();
    p->calculateBill();

    delete p;
    return 0;
}

TEST CASES;

1. IN- PATIENT
Input:
1

```

John

DrSmith

2000

3000

Output:

1.InPatient

2.OutPatient

3.Emergency

4.Critical

Enter choice: Enter Patient Name: Enter Doctor Name: Room Charge: Treatment Cost:

Bill = 5000

2. CRITICAL PATIENT

Input:

4

Alice

DrJohn

2000

3000

1500

Output:

1.InPatient

2.OutPatient

3.Emergency

4.Critical

Enter choice: Enter Patient Name: Enter Doctor Name: Room Charge: Treatment Cost: Emergency Charge:

Total Bill = ₹6500